For anyone familiar with Zork[1], this is along the same lines with a few text prompts to enter the data to set the real-time clock. With Newline again selected in our Serial Monitor, pressing `Y` and Enter will start the process of setting the date and time beginning with the year and working down to the minutes. There are no time limits, so we can take as long as we need to enter each number, but there also is no error correction so the sketch will provide feedback as to what was entered. We start the process in the `setup()` function with a series of statements to print the message. From here we use a `while` loop in the following line to wait until data has been entered:

```
while (!Serial.available()) delay(10);
```

This statement will loop continuously until data has been placed in the serial buffer, signaling user feedback. Once the character `'Y'` or `'y'` has been entered, we begin with the `setTime()` function. This function walks through a series of prompts to input each unit of time. A call to `readByte()` is used to translate the incoming ASCII characters to decimal values, much like we did at the beginning of this chapter, and then assigns that value to the corresponding unit. Once this is done for all seven units, seconds is automatically assigned 0, each value is then converted using a new function `decToBcd()` to write each value to our RTC as BCD values.

When all of this has been completed, the time and date are permanently stored in the RTC, at least until the battery backup is removed or it runs out, and then we are returned to the `setup()` function, where we print the revised date and time and say good-bye. Fairly long and drawn out, at least maybe a bit, but each part of the sketch has a specific job to do to make things a little easier for us to understand. So let's now have a brief look at some of the functions from the Wire library that make all this happen.

Wire Library

The Wire library provides access to a multitude of I2C devices, not just the DS1307 real-time clock, and we used it in Chapter 8 to talk to the BlinkM MinM smart LED. Likewise, while we have typically used analog sensors for input in our projects, these could be replaced with I2C devices that would offer us many benefits that outweigh the added complexity of talking to them. I2C can be used to read from many different devices, such as accelerometers, electronic compasses, and even FM radios. Because every device will have its own requirements, we will only briefly discuss the standard functions of the Wire library here.

begin()

Like other libraries, we need to call the `begin()` function once in our code, in this case to join the I2C data bus. Its syntax depends on whether the Arduino is to be the master or slave device.

```
Wire.begin(address)
```

The address is not needed for the master device, which is usually the Arduino, but there might be reason to set the Arduino as a slave, which would need an address assigned to it. For most things, the address in this parameter is not used.

---

[1] Wikipedia, "Zork," `http://en.wikipedia.org/wiki/Zork`.